

Day 3 - Strings Substring & subsequence

ITSRUNTYM

Substrings vs Subsequences

- **Substring:** Continuous block of characters.
 - e.g., "abc" in "abcde"
- **Subsequence:** Characters taken in order, not necessarily continuous.
 - e.g., "ace" in "abcde"

1. Check if s1 is a subsequence of s2

(Two Pointers)

Idea: Move two pointers on s and t, check if all characters of s appear in t in order.

Code:

```
public boolean isSubsequence(String s, String t) {
    int i = 0, j = 0;
    while (i < s.length() && j < t.length()) {
        if (s.charAt(i) == t.charAt(j)) i++;
        j++;
    }
    return i == s.length();
}
```

C++

```
bool isSubsequence(string s, string t) {
    int i = 0, j = 0;
    while (i < s.length() && j < t.length()) {
        if (s[i] == t[j]) {
```

```

        i++;
    }
    j++;
}
return i == s.length();
}

```

What Are Substring Patterns?

A **substring** is a contiguous block of characters within a string.

Substring problems often require:

- Finding/counting substrings with certain properties.
- Checking if a pattern exists.
- Sliding window usage for optimization.
- Hashing or frequency maps for tracking characters or patterns.

Topics in Substring Patterns

1. All Substrings of a String

Generate and print all substrings.

Example:

For string "abc", the substrings are:
 "a", "ab", "abc", "b", "bc", "c"

Code:

```

public class AllSubstrings {
    public static void main(String[] args) {
        String s = "abc";
        int n = s.length();
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j <= n; j++) {
                System.out.println(s.substring(i, j));
            }
        }
    }
}

```

Time Complexity: $O(n^2)$

C++ Version

```
#include <iostream>
```

```

#include <string>
using namespace std;

int main() {
    string s = "abc";
    int n = s.length();

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j <= n; j++) {
            cout << s.substr(i, j - i) << endl;
        }
    }

    return 0;
}

```

2. Longest Substring Without Repeating Characters

Find the length of the longest substring with all unique characters.

LeetCode: 3. Longest Substring Without Repeating Characters

Approaches:

Brute Force – Check all substrings

- Time: $O(n^3)$

Optimized – Sliding Window + HashSet/Map

- Time: $O(n)$

Code (Optimal – Sliding Window):

```

public int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0;
    int right = 0;
    int maxLen = 0;
    while (right < s.length()) {
        if (!set.contains(s.charAt(right))) {
            set.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
            right++;
        } else {
            set.remove(s.charAt(left));
            left++;
        }
    }
}

```

```

    }
    return maxLen;
}

```

Time: $O(n)$, Space: $O(1)$ for ASCII

C++

```

#include <iostream>
#include <unordered_set>
#include <string>
using namespace std;

int lengthOfLongestSubstring(string s) {
    unordered_set<char> charSet;
    int left = 0, right = 0, maxLen = 0;

    while (right < s.length()) {
        if (charSet.find(s[right]) == charSet.end()) {
            charSet.insert(s[right]);
            maxLen = max(maxLen, right - left + 1);
            right++;
        } else {
            charSet.erase(s[left]);
            left++;
        }
    }

    return maxLen;
}

```

3. Longest Repeating Character Replacement

Replace at most k characters to get the longest repeating substring.

LeetCode: 424. Longest Repeating Character Replacement

Idea:

- Use sliding window to maintain a window where you can replace at most k chars.
- Track the **most frequent char count** in window.

Code:

```

public int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int left = 0;
    int maxCount = 0;
    int maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        count[s.charAt(right) - 'A']++;
        maxCount = Math.max(maxCount, count[s.charAt(right) - 'A']);
        while (right - left + 1 - maxCount > k) {
            count[s.charAt(left) - 'A']--;
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}

```

Time: $O(n)$, Space: $O(1)$

C++

```

#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

int characterReplacement(string s, int k) {
    vector<int> count(26, 0);
    int left = 0, maxCount = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        count[s[right] - 'A']++;
        maxCount = max(maxCount, count[s[right] - 'A']);

        while ((right - left + 1) - maxCount > k) {
            count[s[left] - 'A']--;
            left++;
        }

        maxLen = max(maxLen, right - left + 1);
    }

    return maxLen;
}

```

4. Longest Palindromic Substring

LeetCode: 5. Longest Palindromic Substring

Approaches:

- Brute force: $O(n^3)$
- **Expand around center:** $O(n^2)$

Key Idea: Expand Around Center

- A palindrome is symmetric around its center.
- So we can **expand outward** from every index i as a possible **center**.
- There are **$2n-1$ centers** in a string of length n :
 - n odd-length centers $\rightarrow i == j$
 - $n-1$ even-length centers $\rightarrow j = i + 1$

Main Function:

```
public String longestPalindrome(String s) {
    int start = 0, end = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i);    // Odd-length palindrome
        int len2 = expand(s, i, i + 1); // Even-length palindrome
        int len = Math.max(len1, len2); // Take longer of the two

        if (len > end - start) {
            // Update the start and end pointers of the longest palindrome
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }
    return s.substring(start, end + 1);
}
```

Expand Function:

```
private int expand(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--; // Expand left
        right++; // Expand right
    }
}
```

```
    return right - left - 1; // Total length of palindrome
}
```

C++

```
#include <iostream>
#include <string>
using namespace std;

int expand(const string& s, int left, int right) {
    while (left >= 0 && right < s.length() && s[left] == s[right]) {
        left--;
        right++;
    }
    return right - left - 1; // length of palindrome
}

string longestPalindrome(string s) {
    int start = 0, end = 0;

    for (int i = 0; i < s.length(); i++) {
        int len1 = expand(s, i, i);    // odd-length palindrome
        int len2 = expand(s, i, i + 1); // even-length palindrome
        int len = max(len1, len2);

        if (len > end - start) {
            start = i - (len - 1) / 2;
            end = i + len / 2;
        }
    }

    return s.substr(start, end - start + 1);
}
```

Dry Run Example:

Input: "babad"

Iteration i = 0 → center = 'b'

- len1 = 1 (odd), len2 = 0 (even)
- maxLen = 1 → update start = 0, end = 0

Iteration i = 1 → center = 'a'

- len1 = 3 (odd) from "bab"
- len2 = 0

- $\text{maxLen} = 3 \rightarrow \text{update start} = 0, \text{end} = 2$

Iteration i = 2 $\rightarrow$ center = 'b'

- $\text{len1} = 3$ (odd) from "aba"
- $\text{maxLen} = 3 \rightarrow$ no change (same size)

Final result: $\text{s.substring}(0, 2 + 1) = \text{"bab"}$

Time and Space Complexity

Type	Value
Time	$O(n^2)$ — for each center, expanding could take $O(n)$
Space	$O(1)$ — no extra space used